

Summarized version

- SDG 7 - Affordable and clean energy and SDG 11 - Sustainable cities and communities
- Name of our platform/project
 -
- Problem
 - Households with solar panels have excess electricity that is usually wasted or sold back to the grid at low rates. Meanwhile, nearby households (with or without solar panels) pay high retail prices for the same electricity. This system is expensive and wasteful.
 - P2P trading is allowed in specific, trial-based, or newly emerging contexts, rather than being universally available in all areas. Our platform helps to increase this
- Solution
 - A web-based platform allowing peer-to-peer energy trading, where households can buy and sell solar energy from each other. The transactions are secure and recorded on Solana. This benefits the sellers, solar owning households, as they earn a fairer compensation for their excess energy. It also benefits buyers by allowing them to access electricity at lower rates than grid rates. Overall, communities as a whole become less dependent on centralized providers and prevent wastage of renewable energy.
 - Why Solana? It is a blockchain-designed, high-performance system known for extreme speed and low fees. It includes proof of history and proof of stake to secure the network. Additionally, it doesn't rely on secondary (layer 2) networks for scalability.
- Data
 - Country best suited for - Australia (due to their retail vs selling rates)
 - <https://www.solarchoice.net.au/learn/energy/feed-in-tariffs/#:~:text=Compare%20Now-,How%20much%20are%20Feed%2Din%20Tariffs%20in%202025?,a%20bit%20higher%2C%20some%20lower>

- <https://cyanergy.com.au/blog/everything-you-need-to-know-about-net-metering-in-australia/>
- <https://lenergy.com.au/can-you-sell-solar-power-to-the-grid/>
- https://www.bluettipower.com.au/blogs/buying-guide/selling-electricity-back-to-the-grid-australia-complete-guide-to-solar-feed-in-tariff?srsId=AfmBOopbcZhLd0ycCUKbGLXGnN2PeMDCX9zY7j_K6jVefR-m6iAN6nAP
- <https://www.solarrun.com.au/selling-my-solar-power-to-neighbours-when-will-i-be-able-to-do-it/#:~:text=of%20the%20game!-,Can%20I%20do%20it%20now?,those%20without%20their%20own%20panels.%E2%80%9D>
- <https://solarpowerconference.com/neighbour-to-neighbour-solar-trading-lifts-returns-and-eases-strain-on-the-grid/>
- Key features
- How it works
 - Users (frontend) - 4 pages
 - Listings page → Shows all households selling surplus energy as listing cards to browse. Each listing card shows how much kWh is available, the price and a “buy” button (like Facebook Marketplace)
 - Transaction page - after the user clicks the “buy” button. Shows the summary of the purchase and confirmation to buy (like all common transaction pages). Then the blockchain is triggered.
 - Success page + confirmation (showing the trade went through)
 - My profile - options like “my wallet”, showing token balance, “token history”, showing how much they have bought or sold, etc.
 - Seller (frontend) - 1 page
 - A form-like page for when a household wants to sell, showing their surplus and current market prices
 - Energy Logic (backend) → who has surplus energy and how much
 - We will simulate energy data
 - Household solar production - changes by time of day (peak at noon, zero at night)

- Household consumption - how much they consumed throughout the day
 - Surplus calculation = production - consumption
 - If surplus > 0, then the household can sell
 - If surplus ≤ 0, then the household can buy (if they want more)
- Pricing logic (backend)
 - We will simulate pricing data
 - Price = minimum price + (demand/supply) * factor
 - When fewer people are buying, and little surplus exists, the price goes up automatically. When surplus is high, price drops
- Blockchain layer → You write a smart contract (called a Program in Solana) using the Anchor framework, which manages token balances, validates buy/sell orders, and auto-settles payments when a trade completes. Each energy trade gets written permanently to the ledger. Solana has a built-in token standard called SPL. You create a custom EnergyToken where 1 token = 1 kWh. When House A sells 5 kWh to House B, 5 tokens transfer from B's wallet to A's wallet automatically via the smart contract. No middleman, no manual settlement. Use **Solana Devnet** (a free test network) so you can simulate real blockchain transactions without real money. The simulation engine can use hardcoded or randomized solar/demand data to get started with.
- Database → Blockchain stores transaction records. We still need a regular database for user profiles, household configurations, simulation state, and historical analytics.
 - We will create some profiles with household data for simulation
- Tech Stack
 - Frontend → React + Tailwind CSS + Next js + D3 js for energy flow animations
 - Backend → Node js
 - Blockchain layer → Solana + Anchor framework
 - Database → Firebase or Snowflake

Peer-to-Peer Energy Trading Simulator

A Decentralized Marketplace for Local Renewable Energy

1. Introduction

Renewable energy generation is rapidly increasing as more households install solar panels. However, a major inefficiency exists: **excess electricity generated by homes often goes unused or is sold back to centralized utilities at low prices.**

At the same time, nearby households may need electricity but must buy it at higher rates from the grid.

This project proposes a **Peer-to-Peer (P2P) Energy Trading Simulator**, a platform that allows households to **buy and sell surplus solar energy directly with each other** using a decentralized ledger.

By leveraging blockchain technology such as **Solana**, the system ensures that energy transactions are transparent, secure, and automated without requiring a central authority.

The simulator demonstrates how decentralized technologies could enable **community-based energy markets**, improving renewable energy efficiency and reducing dependence on centralized power grids.

2. Problem Statement

Current electricity distribution systems are highly centralized. Even when homes produce renewable energy, they face several challenges:

Inefficient Energy Distribution

Excess solar energy is often wasted or sold back to utilities at a very low rate.

Lack of Local Energy Markets

Households cannot easily sell energy directly to nearby consumers.

Limited Incentives for Renewable Energy Producers

Solar panel owners often receive less compensation than the retail electricity price.

Grid Strain

Centralized energy systems can experience congestion and inefficiencies when demand fluctuates.

These limitations prevent communities from fully utilizing locally generated renewable energy.

3. Proposed Solution

The proposed platform creates a **digital marketplace where households can trade electricity directly with neighbors.**

Households that generate solar power can list their excess energy for sale, while households that require electricity can purchase energy through the platform.

Each transaction is recorded on a blockchain ledger to ensure transparency and immutability.

The system simulates:

- solar energy generation
- household electricity demand
- energy trading between participants
- dynamic pricing

- transaction recording on a distributed ledger

In the system, energy is represented digitally through tokens.

Example representation:

1 EnergyToken = 1 kWh of electricity

When a household purchases energy, the corresponding tokens are transferred and the transaction is recorded on the blockchain.

4. Objectives

The key objectives of this project are:

- Demonstrate how decentralized systems can support local energy markets
 - Simulate peer-to-peer electricity trading between households
 - Improve utilization of renewable energy within communities
 - Provide transparent and secure energy transactions using blockchain
 - Explore how dynamic pricing can balance supply and demand
-

5. System Architecture

The system consists of three main components.

5.1 User Interface (Frontend)

The frontend dashboard allows users to interact with the system.

Users can:

- View current energy production
- Monitor household energy consumption
- List surplus energy for sale
- Purchase energy from other households
- Track transaction history

The interface visually displays a local energy network where houses exchange power.

Possible technologies:

- React
 - Next.js
 - Tailwind CSS
-

5.2 Simulation Engine (Backend)

The backend simulates real-world energy conditions.

The system models:

- Solar energy production based on time of day
- Household electricity demand
- Available surplus energy
- Energy pricing based on supply and demand

Example simulation rule:

Surplus Energy = Energy Generated – Energy Consumed

If surplus energy is positive, the household can sell electricity on the marketplace.

Possible technologies:

- Node.js

- Python
 - Express API
-

5.3 Blockchain Layer

All transactions are recorded on a decentralized ledger using **Solana**.

A smart contract manages:

- energy token balances
- buy/sell orders
- transaction validation
- automatic payment settlement

Example transaction record:

Seller: House A

Buyer: House B

Energy Sold: 5 kWh

Price per kWh: 0.3 tokens

Total Cost: 1.5 tokens

Timestamp: 14:32

Using blockchain ensures that the system is:

- transparent
 - tamper-proof
 - decentralized
-

6. Key Features

Energy Tokenization

Electricity is represented digitally as tokens, making it easier to trade small units of power.

Peer-to-Peer Trading

Households can buy energy directly from other households instead of relying solely on the grid.

Transparent Transactions

Every trade is recorded on the blockchain ledger.

Dynamic Energy Pricing

Prices can automatically adjust depending on supply and demand.

Example pricing rule:

Energy Price = Base Price + Demand Factor

Real-Time Energy Marketplace

Users can view available energy listings and trade instantly.

7. Example Scenario

Consider a neighborhood with three households:

House A generates solar energy.

House B has high electricity demand.

House C generates moderate solar energy.

During the afternoon:

- House A produces 12 kWh
- House A consumes 7 kWh
- House A has 5 kWh surplus

House A lists 5 kWh on the marketplace.

House B purchases 3 kWh.

The transaction is recorded on the blockchain, and tokens are transferred automatically.

House C may also sell its surplus energy later in the evening.

This allows the neighborhood to share renewable power locally.

8. Potential Impact

The proposed system could contribute to several real-world benefits.

Improved Renewable Energy Utilization

Local trading ensures that excess solar energy is used rather than wasted.

Reduced Energy Costs

Consumers may buy electricity at lower prices compared to utility rates.

Incentives for Solar Adoption

Homeowners gain additional revenue by selling surplus electricity.

Decentralized Energy Infrastructure

Communities become less dependent on centralized energy providers.

9. Future Enhancements

Several advanced features could be added in future versions:

Smart Meter Integration

Real-world smart meters could automatically track energy production and consumption.

AI-Based Energy Forecasting

Machine learning models could predict solar generation and demand.

Battery Storage Integration

Homes with batteries could store energy and sell it during peak demand.

Carbon Impact Tracking

The system could calculate how much CO₂ emissions are reduced through renewable energy trading.

10. Conclusion

The Peer-to-Peer Energy Trading Simulator demonstrates how decentralized technologies can transform the way energy is distributed and consumed.

By enabling households to trade electricity directly with each other, the system creates a **local energy marketplace that encourages renewable energy adoption and improves efficiency.**

Using blockchain platforms such as **Solana**, the project illustrates how secure, transparent, and decentralized infrastructure could support future smart energy networks.

This simulation provides a foundation for exploring **community-driven renewable energy systems and decentralized power markets.**

BACKEND BREAKDOWN: (VERSION #1: BLOCKCHAIN BACKEND ONLY)

1. Database:

1. **Households** — every user. Has their email, password, role (prosumer/consumer/both), credit balance, location, reliability score.
2. **Meter Readings** — energy data logged every 15 minutes per household. Stores generation, consumption, and surplus (generation minus consumption).
3. **Listings** — when a prosumer has surplus they want to sell. Stores how much kWh, at what price, and whether it's active/matched/expired.
4. **Trades** — when a match happens. Stores who bought from who, how much, what price, what the status is, and the Solana transaction hash at the end.
5. **Forecasts** — cached predictions from the AI model. Stores the next 24 hours of predicted surplus per household so you're not running the model on every request.

2. **AUTH** (not clear on this yet cuz of the blockchain thing, whether it'll be through login/email or crypto wallet)

3. Meter Data

Two modes depending on whether you have real hardware:

Synthetic mode (demo) — math formula that mimics a real solar panel. Generation follows a bell curve peaking at noon. Consumption spikes in the morning and evening. The difference is surplus. Adds realistic random noise so it doesn't look fake.

Real mode (production) — calls SolarEdge or Enphase API every 15 minutes. Pulls actual generation and consumption numbers. Stores them identically to synthetic mode so the rest of the system doesn't care which source it came from.

Every reading gets stored in the database with a timestamp. The current surplus endpoint just returns the most recent reading.

4. Listings

When a prosumer has surplus, they list it for sale. A listing has:

- How many kWh available
- Price per kWh
- Status (active, matched, expired)

Listings can be created two ways:

- **Manual** — prosumer clicks "sell my surplus" in the app
- **Automatic** — Celery job detects surplus above a threshold and auto-lists it every 15 minutes

Listings expire end of the day and user can sell back to the grid instead

The market endpoint returns all active listings sorted by best score (proximity + price + reliability).

5. Matching Engine

Runs every 15 minutes as a background job. For every consumer household it finds the best available listing by scoring each one:

Proximity (40%) — closer prosumer means less energy lost in transmission. A neighbor 100 meters away is much better than someone 2km away.

Price (40%) — cheaper is better for the consumer.

Reliability (20%) — has this prosumer actually delivered in the past? Calculated as the ratio of kWh promised vs kWh actually delivered across all past trades. Starts at 1.0, updates after every trade.

Takes the highest scoring listing, creates a trade, deducts credits from consumer immediately.

6. Trades

Three stages every trade goes through:

Matched — buyer and seller paired. Consumer's credits are reserved (deducted immediately so they can't double-spend). Listing marked as matched so nobody else buys it.

Delivered — oracle confirms energy actually arrived. This is when the meter is checked again to verify the prosumer actually had the surplus they promised. If they delivered less, payment is pro-rated. If they delivered nothing, consumer gets full refund.

Settled — credits transferred to prosumer, Solana transaction recorded, reliability score updated. Trade is done.

If a trade fails at any point — credits returned to consumer, listing goes back to active, prosumer reliability score takes a small hit.

7. AI Forecasting (LSTM Model)

Predicts each household's surplus for the next 24 hours. Used for two things:

- Showing prosumers "you'll have 8.2 kWh tomorrow, list it now?"
- Helping consumers pre-book cheap energy before peak demand

What it learns from:

- Past 7 days of generation and consumption readings
- Time of day and day of week patterns
- Weather forecast (cloud cover, temperature) from Open-Meteo API

How it works: LSTM is a type of neural network that's good at sequences — it remembers patterns over time. You feed it the last 7 days and it predicts the next 24 hours.

Training: Train it on synthetic data first (you control the patterns so you know if it's learning correctly). Retrain every week via GitHub Actions as real data accumulates.

Output: 24 hourly surplus predictions with a confidence range. Stored in the forecasts table so the API just reads cached predictions instead of running the model on every request.

8. Solana Settlement (Background)

Runs silently after every trade settles. User never sees or touches this.

What it does: Takes the verified kWh delivered and payment amount, signs it with the oracle's private key, submits it to the Solana smart contract on devnet. Contract records it permanently on-chain.

Why it matters: Every trade now has a tamper-proof on-chain receipt. Nobody — not even you — can alter it. Users can optionally click "view blockchain proof" in their history to see it on Solana Explorer.

The oracle: A signing service that bridges real-world meter data to the blockchain. The smart contract only trusts readings signed by the oracle's specific keypair. This prevents anyone from submitting fake delivery confirmations.

9. Background Jobs (Celery)

Three jobs running automatically:

Every 15 minutes:

- Poll all meters (or generate synthetic readings)
- Detect surplus above threshold
- Auto-create listings for prosumers
- Run matching engine for consumers
- Trigger any pending settlements

Every night:

- Expire listings older than 30 minutes that never got matched
- Calculate monthly credit totals per household
- Trigger Solana batch settlement for the day's trades
- Cache fresh forecast predictions

Every Sunday:

- Retrain LSTM model on the past week's new data
- Update saved model weights
- Validate accuracy before replacing old model

10. Claude Monthly Report

At the end of each month, sends each user's stats to Claude API and gets back a plain-English summary:

Stats sent:

- Total kWh sold/bought
- Credits earned/spent
- Carbon offset (kWh × 0.42kg CO2)
- Best trading day
- Reliability score trend

What Claude writes: A 100-word friendly summary. Something like "Great month — you powered 3 neighbors and offset the equivalent of driving 180km. Your reliability score hit a new high."

Stored in the database so it loads instantly when user opens their history page.

#HOW IT WORKS:

User signs up (Auth)



Meter starts reading every 15min (Meter Data)



Surplus detected → listing created (Listings)



Matching engine finds buyer (Matching)



Trade created, credits reserved (Trades)



Oracle verifies delivery (Oracle)



Credits transferred, Solana records it (Solana)



LSTM updates next prediction (Forecasting)



End of month → Claude writes report (Reports)

Setup

- Create FastAPI project structure
- Install dependencies (fastapi, uvicorn, sqlalchemy, psycopg2, celery, redis, torch, anthropic)
- Set up .env file with DB URL, JWT secret, Anthropic key, Solana RPC
- Run PostgreSQL via Docker
- Connect SQLAlchemy to database

Database

- Define 5 models: Household, MeterReading, Listing, Trade, Forecast
- Run `Base.metadata.create_all()` to create tables
- Test connection works

Auth

- Write password hashing with bcrypt
- Write JWT token creation + decoding
- Build `/auth/signup` endpoint
- Build `/auth/login` endpoint
- Build token middleware that protects all other routes

Meter Data

- Build synthetic generator (solar curve formula)
- Store readings every 15min per household
- Build `/households/{id}/surplus` endpoint (returns latest reading)
- Build `/households/{id}/readings` endpoint (returns last 24hrs)

Listings

- Build `/listings/create` endpoint
- Build `/listings/market` endpoint (all active listings)
- Add 30-minute auto-expiry logic
- Add auto-listing trigger when surplus exceeds threshold

Matching Engine

- Write scoring function (proximity + price + reliability)
- Write `find_best_match()` for a given consumer
- Test matching logic with dummy data

Trades

- Build `/trades/buy` endpoint
- Deduct credits from consumer on buy
- Build `/trades/{id}/settle` endpoint
- Add pro-rated payment logic (partial delivery)
- Add reliability score update after every settlement
- Build `/trades/{id}/history` endpoint
- Handle failed trade → refund consumer → restore listing

Background Jobs (Celery)

- Set up Celery with Redis as broker
- Write job: poll meters every 15min
- Write job: run matching engine every 15min
- Write job: expire old listings nightly
- Write job: retrain LSTM model weekly
- Test all jobs run on schedule

AI Forecasting

- Build LSTM model architecture
- Generate synthetic training data
- Train model, save weights
- Write `predict_surplus()` function
- Cache predictions in Forecasts table
- Build `/households/{id}/forecast` endpoint

Solana Settlement

- Set up oracle keypair
- Write `settle_on_chain()` function
- Call it silently after every trade settles

- Store tx signature in Trade record
- Build `/trades/{id}/proof` endpoint (returns Solana Explorer link)

Claude Report

- Write stats aggregation query (kWh, credits, carbon)
- Call Claude API with stats
- Store report in database
- Build `/reports/{id}/monthly` endpoint

Testing

- Test every endpoint with Postman or HTTPie
- Simulate full trade lifecycle end to end
- Run matching engine on 10 synthetic households
- Verify Solana tx signatures appear in trade records
- Verify Claude report generates correctly

Deploy

- Dockerize FastAPI + Celery + Redis
- Push to Railway
- Set all env variables in Railway dashboard
- Test live endpoints
- Don